泰坦尼克号生存预测 - 学习笔记

一、处理数据

1. 读取

你有数据，第一步是让机器可以读取，这里导入pandas（记作pd方便后续使用，是用来管表格数据清洗、预处理的），读取给定的数据。

读取：

```
pd.read_csv（路径）
```

利用pandas读取后打印出来发现Age和Cabin两项有缺失

2. 清洗

题目要求预测乘客幸存概率，注意到编号、姓名、船票号对生存预测没帮助（完全无关或过于杂乱），Cabin缺失太多（缺失率超70%），于是将他们删去；年龄一项有缺失（较少），可以用平均值补上；港口用出现最多的补上（众数）

众数：

```
.mode()[0]
```

3. 特征工程

机器看不懂一些表达，把它变成可读懂的（str不行，int64、float64 或 bool可以）。

- **性别**：把 male 变成 0，female 变成 1（map:翻译规则）；
- **登船港口**：C、Q、S 变成三列 0和1 (独热编码get_dummies：数字无大小之分，仅为分类)，这里删去了第一列（不是数据哈），用（1,0）、（0,1）、（0,0） ➡️C港口即可表示完

但是为什么要这么费事的删去第一列呢？ 为什么要加这个 drop_first=True？ 在机器学习里，如果不删这一列，这三列的数字加起来永远等于1（比如1+0+0=1）。这种"完全相关"会让模型算矩阵乘法时产生"多重共线性"问题，导致算出错误的结果。删掉一列，就相当于把密码变成"缺省暗号"，模型一样能推断出C，而且更安全。

4. 代码学习

- 在 Pandas 里，axis=0 代表"行"，axis=1 代表"列"；
- r'...' 是告诉电脑，后面的斜杠 \ 别瞎翻译，原样照搬，防止路径报错；
- fillna：Fill Not Available，填补空缺；
- data['Age'].mean()：算出所有乘客的平均年龄，然后把它填进空洞里；
- mode()[0]：找出现次数最多的那个值（众数）；
- map：映射。
- .values：把 Pandas 表格格式，转换成普通的 Numpy 数组（去掉表格的索引和列名，纯粹变成一串数字，PyTorch才认识 ）。

- [:, 2]：这是 Numpy 的切片语法，逗号前面是"行"，逗号后面是"列"。
    - : 代表"所有行"；
    - 2 代表"第 2 列"（索引从 0 开始）

二、划分数据

我按照8训练2测试来划分的； 把数据转成NumPy数组：把表格的皮剥掉，变成光秃秃的数字矩阵。 把"是否幸存"这一列单独拿出来叫 y（答案），剩下的8列叫 X（线索）。

三、搭建神经网络

1.

造两个外卖员：用 Dataset 和 DataLoader 把数据打包成32个一盒的小外卖。训练集外卖员每次打乱顺序送（防死记硬背），测试集外卖员不打乱送（保持公平）。

2.

造大脑：用 nn.Module 造一个"8进、16中间、1出"的大脑。 先搞两个 nn.Linear（算账机器）负责做乘法和加法。 中间插一个 nn.ReLU（情绪过滤器）把负数变0，让大脑能拐弯。 最后加 nn.Sigmoid（挤压机），把结果强行变成0~1之间的概率。

3.

定奖惩机制：请出 nn.BCELoss（打分器）负责算错了多少，请出 Adam（改错老师）负责改错，步伐定在 0.001。

四、训练循环

复习100轮。前向计算 打分器算错（loss = criterion）。

把旧错清空（zero_grad）。

反思错在哪（loss.backward() 自动求导）。

改错老师微调大脑（optimizer.step()）。

每10轮打印一次战报，看看犯错率降了没，准确率升了没。

五、评估

每复习一轮，就用保险柜里的179份"期末考卷"考一次试，算出准确率。 100轮跑完，把大脑学到的所有知识存成一个文件：titanic_model.pth。 把"当时算的平均年龄、众数港口、列的顺序"存成 preprocess_info.json。 画两张图（损失曲线、准确率曲线）放进 docs 文件夹。

六、新样本预测

input() 交互 注意提示输入者输入的格式

七、问题

1.前面都算了age平均和embarked众数，这两行意义何在?

```
data['Age'] = data['Age'].fillna(data['Age'].mean())
data['Embarked'] = data['Embarked'].fillna(data['Embarked'].mode()[0])
```

```
train_age_mean = X_train[:, 2].mean()
train_embarked_mode = 'S'
```

前面算平均只是为了填满表格，算完就没了；这里再算平均值是为了存到内存里，如果再有缺年龄的新对象直接用这个填上

2.为什么不直接把 Pandas 表格塞给 torch.tensor？非要中间加个 .values 转成 Numpy?

直接把 Pandas 塞给 torch.tensor 也能跑。但中间加个 .values 转成 Numpy，是为了让流水线最顺畅、最不容易报错。train_test_split（来自 sklearn）是专门用来切数据的，它最喜欢吃 Numpy 数组。它吃 Pandas 表格虽然也能消化，但会保留行号、列名等没用的外套，切起来容易卡壳。Pandas 表格里混着 int64、float64、bool 等类型，如果直接扔给 torch.tensor，PyTorch 会认不出，PyTorch 的模型只吃 “Tensor（张量）”。

3.shuffle=True的使用：

- 训练集设为 True：每轮训练前打乱数据顺序，防止模型“死记硬背”数据的排列顺序，确保模型每次看的数据都不一样，从而学到真正的规律，而不是数据的排列顺序，增强模型泛化能力。
- 测试集设为 False：测试集只需客观评价模型能力，不需要打乱，保持固定顺序便于复现和评估稳定性。

4.其他细节

- **init**（初始化）：接收外部切分好的特征（X_train）和标签（y_train）。并做好类型转换。
- **len**（报库存）：返回数据集的总样本数量。DataLoader 依赖此方法得知数据总量，以决定批次划分。
- **getitem**（按号取货）：给定索引 idx，返回对应的单条特征和标签。这是 DataLoader 批量取数据的基础单元。
- batch_size=32：设置每次送入模型的数据量为32条。批量训练是深度学习的基础，可以避免一次性加载所有数据导致内存溢出，并有效利用计算资源。

数据流转与形态演变全流程

步骤	数据形态 / 数据类型	代码操作（对应行号）	形状 (Shape)	大白话备注
1. 读取原始数据	`pandas.DataFrame` (含 int/float/str/bool)	`pd.read_csv('train.csv')`	`[891, 12]`	带行号、列名，含有文字和缺失值

2. 清洗与编码	`pandas.DataFrame` (纯数字/布尔)	`data.drop()`
		`fillna()`
		`map()`
		`get_dummies()`
			`[891, 9]`	删掉无用列，填补缺失值，文字变数字

3. 特征标签分离	`numpy.ndarray` `float32`	`y = ...values.astype(np.float32)` `X = ...drop().values.astype(np.float32)`	`y:[891]` `X:[891, 8]`	把答案(y)和线索(X)分开，并剥掉表格外皮

4. 切分数据集	`numpy.ndarray`
	`float32`
		`train_test_split(X, y, test_size=0.2)`	`X_train:[712, 8]`
			`y_train:[712]`
			`X_test:[179, 8]`
			`y_test:[179]`
				按 8:2 切分。固定 `SEED=33` 保证复现

步骤	数据形态 / 数据类型	代码操作（对应行号）	形状 (Shape)	大白话备注
5. 封装 Dataset	`torch.Tensor float32`	`self.features = torch.tensor(features, dtype=torch.float32)` `self.labels = torch.tensor(labels, dtype=torch.float32).unsqueeze(1)`	`X_train_tensor: [712, 8]` `y_train_tensor: [712, 1]`	转成 PyTorch 专属张量。标签用 `unsqueeze(1)` 升维对齐
6. DataLoader 打包	`torch.Tensor float32`	`DataLoader(batch_size=32, shuffle=True)`	每次拿到：`features:[32, 8]` `labels:[32, 1]`	外卖员每批送 32 份，打乱顺序防死记硬背
7. 送入模型运算	`torch.Tensor float32`	`y_pred = model(features)`	模型输出：`y_pred:[32, 1]`	前向计算，算出 32 个乘客的生存概率

数据流转逻辑总结：

> 3. **DataLoader 的作用是什么？** 它把整个大张量切成一个个 `batch_size=32` 的小批次，并在训练集上自动打乱顺序（`shuffle=True`），确保模型每次看的数据都不一样，从而学到真正的规律，而不是数据的排列顺序。

八、模型结构

```
def __init__(self, input_size):
    super().__init__()
    self.linear1 = nn.Linear(input_size, 16)
    self.relu = nn.ReLU()
    self.linear2 = nn.Linear(16, 1)
    self.sigmoid = nn.Sigmoid()

def forward(self, x):
    x = self.linear1(x)
    x = self.relu(x)
    x = self.linear2(x)
    x = self.sigmoid(x)
    return x
```

输入层接收8个原始线索，隐藏层按比例乘一乘、加一加，算出 16个中间特征。把小于0的数变成0，给模型加入“非线性”，让它能学习复杂的曲线规律。输出层把刚才的16个中间特征，浓缩成 把那个得分强行压进 0~1 之间，变成“生还概率”。

九. 损失函数与优化器

损失函数：选择 `nn.BCELoss()`**（二元交叉熵损失）**。二分类任务中，该损失函数能够准确衡量预测概率与真实标签（0或1）之间的差异，配合 Sigmoid 输出层使用效果最佳。

优化器

- 选择 `torch.optim.Adam`。Adam 优化器具有自适应调整学习率的能力，收敛速度快，鲁棒性强。
- 优化器接管了模型所有参数 `model.parameters()`，通过反向传播计算出的梯度来自动更新权重。

十. 主要超参数配置

超参数名称	设定值	说明与实验依据
学习率 (lr)	`0.001`	Adam 优化器的默认黄金起步值，步伐适中，防止震荡或收敛过慢。
批次大小 (batch_size)	`32`	每次送入模型 32 条数据，平衡内存占用与训练稳定性。
训练轮次 (epochs)	`100`	经验值，确保模型在 891 条数据上能充分收敛，而不至于过拟合。
隐藏层神经元数	`16`	隐藏层维度，大于输入特征数，提供足够的非线性拟合能力。
数据集划分比例	`8 : 2`	80% 作为训练集，20% 作为测试集（`test_size=0.2`）。
随机种子 (SEED)	`33`	统一锁定数据划分、模型初始化等随机性，确保实验可复现。

十一. 运行环境配置

- **操作系统**：Windows
- **Python环境**：Anaconda base 环境（Python 3.14.6）
- **深度学习框架**：PyTorch 2.14.1 (CPU版本)
- **依赖库**：Pandas 3.0.3、NumPy 2.4.6、Matplotlib 3.11.0、Scikit-learn 1.9.0

十二、测试集准确率、实验结果分析与心得体会

1. 测试集准确率 本项目使用测试集准确率（Accuracy）作为主要评估指标。经过 100 轮（Epoch）训练后，模型在 179 条测试集数据上的最终表现如下：

- **最终测试集准确率**：**78.77%**（即 179 * 0.7877 ≈ 141 条预测正确）。
- **最终训练集准确率**：82.72%。
- **准确率计算公式**：`Accuracy = 预测正确的样本数 / 测试集样本总数 × 100%`。

2. 实验结果分析 结合训练过程中生成的 `loss_curve.png`（损失趋势图）与 `accuracy_curve.png`（准确率曲线图），可以进行如下分析：

- **损失趋势**：损失值（Loss）从第 1 轮的 0.57 左右稳步下降至第 100 轮的 0.42 左右。曲线呈现出平滑的下降趋势并趋于收敛，说明模型确实从数据中学习到了规律，正在逐步减少预测误差。
- **准确率趋势**：训练集准确率从 71% 稳步上升至 82.72%；测试集准确率从 67% 上升至 78.77%，两者均呈现上升趋势。虽然测试集准确率略低于训练集，但两者差距（约 4%）并不悬殊，且没有出现"训练集准确率一直涨、测试集反而掉下来"的现象。
- **结论**：模型未发生严重的过拟合（Overfitting），泛化能力处于合理范围。**78.77% 的准确率在泰坦尼克号经典数据集上属于正常水平**（该数据集由于特征维度有限，常见准确率通常在 75%-85% 之间），模型表现符合预期。

3. 实验心得与体会 通过本次大作业，我从零开始完成了一个完整的 PyTorch 深度学习项目，主要收获如下：

- **跨越了"环境配置"的鸿沟**：从最初被 VS Code 环境混淆、路径报错、包缺失（如 `No module named 'torch'`）反复折磨，到后来学会区分系统 Python 与 Anaconda 环境，理解绝对路径的作用。这让我明白，**环境配置是程序员的第一道门槛，也是极其重要的基本功**。
- **理解了"先切分，后处理"的重要性**：在数据预处理时，我严格采用了"先划分训练集和测试集，再基于训练集计算均值、众数"的流程，并将这些参数保存到 `preprocess_info.json` 中，确保预测新样本时复用同一套处理方式。这是防止"数据泄露"的核心工程规范。
- **掌握了 PyTorch 的"训练模板"**：学会到了 `Dataset`（数据打包员）、`DataLoader`（外卖员）、`nn.Module`（模型毛坯房）和训练循环（前向 -> 损失 -> 清零 -> 反向 -> 更新）的联动关系。这套模板不仅适用于二分类，换一个数据集和任务同样可以复用。